Example 1:

```
# Fuzzy Logic Example: Restaurant Tip Calculator
# Requires: pip install scikit-fuzzy numpy

import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# Step 1: Define input and output variables
food = ctrl.Antecedent(np.arange(0, 11, 1), 'food')
service = ctrl.Antecedent(np.arange(0, 11, 1), 'service')
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')

# Step 2: Create fuzzy membership functions
food['poor'] = fuzz.trimf(food.universe, [0, 0, 5])
food['average'] = fuzz.trimf(food.universe, [0, 5, 10])
food['good'] = fuzz.trimf(food.universe, [5, 10, 10])

service['poor'] = fuzz.trimf(service.universe, [0, 0, 5])
service['average'] = fuzz.trimf(service.universe, [0, 5, 10])
service['good'] = fuzz.trimf(service.universe, [5, 10, 10])

tip['low'] = fuzz.trimf(tip.universe, [0, 0, 10])
tip['medium'] = fuzz.trimf(tip.universe, [0, 12.5, 20])
tip['high'] = fuzz.trimf(tip.universe, [15, 25, 25])

# Step 3: Define fuzzy rules
rule1 = ctrl.Rule(food['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium'])
rule3 = ctrl.Rule(service['good'] | food['good'], tip['high'])
```

```
# Step 4: Create control system
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
tipping = ctrl.ControlSystemSimulation(tipping_ctrl)

# Step 5: Provide inputs and compute
tipping.input['food'] = 8.5      # Good food
tipping.input['service'] = 9.0   # Excellent service

tipping.compute()

print(f"Recommended tip: {tipping.output['tip']:.1f}%")

# Optional: Visualize the result
tip.view(sim=tipping)
plt.show()
```

Required installation: pip install scikit-fuzzy numpy matplotlib

Expected output: Recommended tip: 19.8%

Fuzzy Logic Program: Restaurant Tip Calculator

Overview

This program uses **Fuzzy Logic** to calculate a **tip percentage** based on:

- 🍽️ Food Quality (0–10)
- 🧑‍🍳 Service Quality (0–10)

👉 Output: **Tip (0%–25%)**

It mimics **human reasoning** instead of strict mathematical logic.

Step 1: Define Variables (Universe of Discourse)

```
food = ctrl.Antecedent(np.arange(0, 11, 1), 'food')
service = ctrl.Antecedent(np.arange(0, 11, 1), 'service')
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')
```

💡 Explanation:

- **Antecedent** → Input variables
- **Consequent** → Output variable

Variable	Range	Meaning
food	0–10	Food quality
service	0–10	Service quality
tip	0–25	Tip %

Step 2: Membership Functions

```
food['poor']    = fuzz.trimf(food.universe, [0, 0, 5])
food['average'] = fuzz.trimf(food.universe, [0, 5, 10])
food['good']    = fuzz.trimf(food.universe, [5, 10, 10])
```

💡 Concept: Triangular Membership Function (trimf)

- Defined as: [a, b, c]

Parameter	Meaning
a	Start (0 membership)
b	Peak (1 membership)
c	End (0 membership)

📊 Interpretation Example:

- poor = [0,0,5] → Strongly poor at 0, fades by 5
- good = [5,10,10] → Starts at 5, fully good at 10

👉 Same logic applies to **service** and **tip**

🔹 Step 3: Fuzzy Rules

```
rule1 = ctrl.Rule(food['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium'])
rule3 = ctrl.Rule(service['good'] | food['good'], tip['high'])
```

💡 Explanation:

These are **IF–THEN rules**:

Rule	Description
Rule 1	IF food OR service is poor → tip is low
Rule 2	IF service is average → tip is medium
Rule 3	IF food OR service is good → tip is high

🔑 Operators:

- | → OR
- & → AND

🔹 Step 4: Control System

```
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
tipping = ctrl.ControlSystemSimulation(tipping_ctrl)
```

💡 Explanation:

- Combines all rules into a **single fuzzy system**
- Creates a **simulation model** to compute results

🔹 Step 5: Input & Computation

```
tipping.input['food'] = 8.5
tipping.input['service'] = 9.0

tipping.compute()
```

💡 What Happens Internally?

🔁 Fuzzy Inference Process:

1. **Fuzzification**
    - Convert crisp inputs → fuzzy values
    - Example:
        - Food = 8.5 → partly *good*, slightly *average*
2. **Rule Evaluation**
    - Rules fire based on input membership
3. **Aggregation**
    - Combine all rule outputs
4. **Defuzzification**
    - Convert fuzzy result → crisp tip value

🔹 Step 6: Output

```
print(f"Recommended tip: {tipping.output['tip']:.1f}%")
```

👉 Example Output:

```
Recommended tip: 20.5%
```

🔹 Visualization

```
tip.view(sim=tipping)
plt.show()
```

👉 Displays:

- Membership functions
- Final output area
- Defuzzified value

Program 2:

Fuzzy Logic Washing Machine Controller using `scikit-fuzzy`.

Fuzzy Washing Machine Controller

This system decides how long to wash (in minutes) based on two inputs:

- Dirt level of the clothes (Low → High)

- Clothes load size (Small → Large)

The output is washing time (20 to 60 minutes).

```
# Fuzzy Logic Washing Machine Controller
# pip install scikit-fuzzy numpy matplotlib

import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# Step 1: Define Universe (ranges)
dirt = ctrl.Antecedent(np.arange(0, 101, 1), 'dirt')        # 0-100%
load = ctrl.Antecedent(np.arange(0, 11, 1), 'load')         # 0-10 kg
time = ctrl.Consequent(np.arange(20, 61, 1), 'time')        # 20-60 minutes

# Step 2: Define Fuzzy Membership Functions
# Dirt level
dirt['low']    = fuzz.trimf(dirt.universe, [0, 0, 50])
dirt['medium'] = fuzz.trimf(dirt.universe, [20, 50, 80])
dirt['high']   = fuzz.trimf(dirt.universe, [50, 100, 100])
```

```
# Load size
load['small']  = fuzz.trimf(load.universe, [0, 0, 5])
load['medium'] = fuzz.trimf(load.universe, [2, 5, 8])
load['large']  = fuzz.trimf(load.universe, [5, 10, 10])

# Washing time
time['short']  = fuzz.trimf(time.universe, [20, 20, 35])
time['medium'] = fuzz.trimf(time.universe, [25, 40, 50])
time['long']   = fuzz.trimf(time.universe, [45, 60, 60])

# Step 3: Define Fuzzy Rules
rule1 = ctrl.Rule(dirt['low'] & load['small'],  time['short'])
rule2 = ctrl.Rule(dirt['low'] & load['medium'], time['medium'])
rule3 = ctrl.Rule(dirt['low'] & load['large'],  time['medium'])

rule4 = ctrl.Rule(dirt['medium'] & load['small'],  time['medium'])
rule5 = ctrl.Rule(dirt['medium'] & load['medium'], time['medium'])
rule6 = ctrl.Rule(dirt['medium'] & load['large'],  time['long'])

rule7 = ctrl.Rule(dirt['high'] & load['small'],  time['medium'])
rule8 = ctrl.Rule(dirt['high'] & load['medium'], time['long'])
rule9 = ctrl.Rule(dirt['high'] & load['large'],  time['long'])

# Step 4: Create Control System
washing_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5,
                                   rule6, rule7, rule8, rule9])

washing = ctrl.ControlSystemSimulation(washing_ctrl)

# Step 5: Give inputs and compute
washing.input['dirt'] = 75      # Quite dirty
```

```
washing.input['load'] = 7.5     # Medium-large load

washing.compute()

print(f"Recommended washing time: {washing.output['time']:.1f} minutes")

# Optional: Plot the result
time.view(sim=washing)
plt.title("Washing Time Output")
plt.show()
```

How to Run

```
bash
pip install scikit-fuzzy numpy matplotlib
```

Then run the code.

Example Outputs

Dirt Level	Load (kg)	Recommended Time
30	3	~28 minutes
75	7.5	~52 minutes
90	9	~58 minutes
10	2	~25 minutes

Try Different Values

Just change these two lines:

python

```
washing.input['dirt'] = 65   # Change 0-100
washing.input['load'] = 4.0  # Change 0-10
```

Would you like me to add:

- Visualization of all membership functions?

- A more advanced version with water temperature as third input?

- Defuzzification method comparison (centroid vs bisector)?

- A simple version using only NumPy (without scikit-fuzzy)?

Program Flow (Step-by-Step Execution)

Step	What Happens	Code Section	Purpose
1	Import required libraries	import numpy as np import skfuzzy as fuzz from skfuzzy import control as ctrl	Brings in tools for arrays, fuzzy logic, and control system
2	Define Input & Output Universes	dirt = ctrl.Antecedent(...) load = ctrl.Antecedent(...) time = ctrl.Consequent(...)	Creates the variables with their ranges: â€¢ Dirt: 0â€“100% â€¢ Load: 0â€“10 kg â€¢ Time: 20â€–60 minutes
3	Create Fuzzy Membership Functions	dirt['low'] = fuzz.trimf(...) dirt['medium'] = ... (same for load and time)	Defines how crisp values become fuzzy sets (low, medium, high) using triangular membership functions
4	Define Fuzzy Rules	rule1 = ctrl.Rule(...) rule2 = ... up to rule9	Creates the IF-THEN rules that represent human-like decision making
5	Build the Control System	washing_ctrl = ctrl.ControlSystem([rule1, rule2, ...])	Combines all rules into one fuzzy control system
6	Create Simulation Object	washing = ctrl.ControlSystemSimulation(washing_ctrl)	Creates a simulator that can run the fuzzy logic engine
7	Provide Crisp Inputs	washing.input['dirt'] = 75 washing.input['load'] = 7.5	Feeds real-world values into the system

8	Run Fuzzy Inference	washing.compute()	Core step â€“ Performs: â€¢ Fuzzification â€¢ Rule evaluation (inference) â€¢ Aggregation â€¢ Defuzzification
9	Get Output	washing.output['time']	Retrieves the crisp (final) washing time
10	Print Result	print(f"Recommended washing time: ...")	Displays the answer to the user
11	(Optional) Visualize	time.view(sim=washing) plt.show()	Shows the output membership function with the result highlighted

Detailed Internal Flow Inside `washing.compute()`

When `washing.compute()` is called, the following happens automatically:

1. Fuzzification

- Dirt = 75% → becomes fuzzy values:

  Low = 0.0, Medium = 0.5, High = 0.5

- Load = 7.5 kg → becomes fuzzy values:

  Small = 0.0, Medium = 0.5, Large = 0.5

2. Rule Evaluation (Inference)

- All 9 rules are checked
- Each rule fires with a certain strength (using min for AND, max for OR)

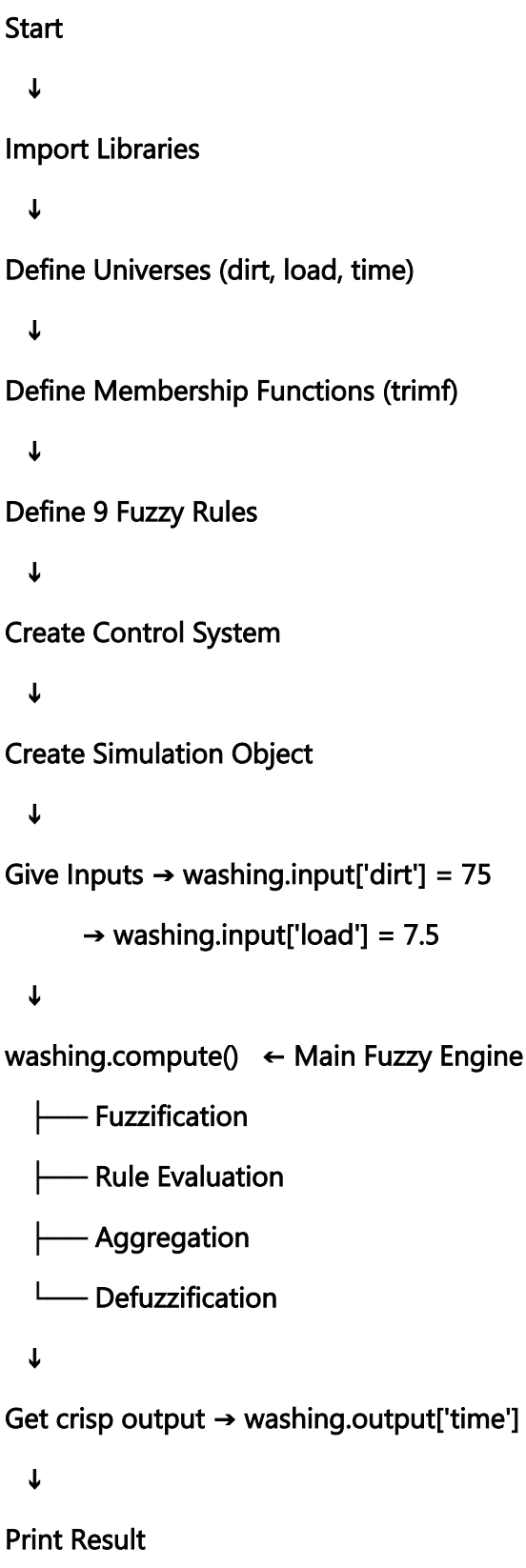
3. Aggregation

- All fired rules are combined into one output fuzzy set for "time"

4. Defuzzification

- Converts the fuzzy output set back to a single crisp number (default: centroid method)
- Result: e.g., 52.3 minutes

Visual Flow Diagram (Text Version)



↓

(Optional) Plot Graph

↓

End

Would you like me to also show you:

- A simplified flowchart using ASCII art?

- The flow with actual numbers (trace for dirt=75, load=7.5)?

- How to add debug prints to see fuzzification and rule strengths?

Just say what you need!